

# Detekce kolidujících překážek a navigace UGV

Shrnutí návrhové diskuse: kolizní modul **Colider**, vrstvená architektura, vyhlazení cesty a reprezentace prostředí (occupancy / ESDF / TSDF).

## 1 · Kontext a cíl

Robot (UGV) drží v paměti seznam překážek (souřadnice + odhad velikosti). Podle aktuálního stavu (pozice, směr, rychlost, úhlová rychlost — z EKF, včetně kovariance) a aktuálního řízení z regulátoru (požadovaná rychlost, úhlová rychlost, doba brzdného zásahu do zastavení) je potřeba vyfiltrovat překážky, které mohou robota v blízké budoucnosti ovlivnit — a to **dost brzy, aby stihl zabrzdit**. Výstupem je obohacený, seříděný seznam hrozeb pro regulátor.

Přístup: z odhadu stavu a řízení namodelovat budoucí trajektorii, kolem ní postavit zametený koridor nafouknutý o obal robota, velikost překážky a nejistotu ( $k \cdot \sigma$ ), a proti němu překážky otestovat.

## 2 · Kolizní modul Colider (implementováno)

Samostatný, přenositelný, čistě metrický modul (namespace **Colider**, bez závislosti na GeoPoint/OSM). Úhly matematicky, v radiánech (CCW od osy +X). Cíleno na net9.0, C#, xUnit; pokryto testy (TDD).

### 2.1 MotionArc

Jeden úsek dráhy s konstantní křivostí — buď rovný (kapsle), nebo kruhový oblouk (výsek mezikruží). Metoda **Project(bod)** v uzavřeném tvaru ( $O(1)$ ) vrátí kolmý odstup od osy, vzdálenost podél dráhy, čas a  $\sigma$  v nejbližším bodě. Konce jsou přirozeně zaoblené ořezem projekce na koncovou pózu. Test „bod je v koridoru“ = `Project(p).Lateral <= w`.

### 2.2 TrajectoryPredictor

Unicycle model rozložený na málo úseků MotionArc: fáze jízdy (rychlost  $vc$ ) + fáze brzdění do zastavení (reprezentativní průměrný poloměr z rychlosti  $vc/2$  — délka i změna směru sedí). Horizont je natažen za bod posledního možného zabrzdění:

```
horizont = max( brzdná_dráha + reakční_dráha + rezerva,
                MinHorizonMeters,
                vc * MinHorizonSeconds )
```

Nejistota roste podél dráhy:  $\sigma(t) = \sigma_0 + \text{growth} \cdot t$ , kde  $\sigma_0$  = poziční  $\sigma$  z EKF.

### 2.3 ObstacleCollisionDetector

Pro každou překážku promítne střed na každý úsek a porovná kolmý odstup s nafouknutým polokoridorem  $w = \text{obal\_robota} + \text{velikost\_překážky} + k \cdot \sigma$ . Metriky se počítají z bodu nejbližšího přiblížení.

Závažnost:

- **Unavoidable** — na kolizním kurzu blíž než brzdná dráha (robot už nezastaví).
- **Imminent** — na kolizním kurzu v horizontu, ale ještě stihne zabrzdit.
- **Watch** — označeno jen kvůli nafouknutí o  $k \cdot \sigma$ ; nominálně je odstup kladný.

Výsledek je seříděn dle závažnosti (sestupně) a času do kolize (vzestupně).

### 2.4 Od vzorkování k analytickému oblouku

Původní verze vzorkovala trajektorii na ~25 kapslí a pro každou překážku je alokovala v cyklu —  $O(\text{překážky} \times \text{segmenty})$  alokací i času. Analytický přístup postaví jen ~2 úseky (jízda + brzdění) jako readonly struct bez alokace; test je pár aritmetických operací na překážku. Výsledek: přesnější geometrie

(žádná diskretizační chyba) a čistší metriky.

## 2.5 Zrychlení a proč se to komplikuje

Při dopředné i úhlové akceleraci se z oblouků stanou spirály (heading kvadratický v čase → Fresnelovy integrály), a hlavně nejbližší bod křivky přestane mít uzavřený tvar. Konstantní křivost je „sladký bod“, kde je Project  $O(1)$ . Zobecnění se proto dělá v predictor (dělení na víc oblouků), ne novým primitivem; MotionArc zůstává beze změny.

### 3 · Vrstvená navigační architektura

Regulátor je zpětnovazební zákon na jeden tik (z aktuálního stavu + cíle vrátí jeden zásah).

Předpokládanou trajektorii proto získáš jeho **rolloutem** (simulace dopředu) — a ta je věrná včetně dynamiky, takže řeší i problém spirál. Doporučená vrstvená smyčka:

- **1. Globální trasa** — waypointy z mapy (GoalField / D\*).
- **2. Rychlý gate** — predikuj dráhu k dalšímu WP a otestuj volnost koridoru (nejdřív levný jeden MotionArc z aktuálního zásahu, případně plný rollout). Koridor nafoukni o  $k \cdot \sigma$  z EKF.
- **Volno -> jed'** (regulátor řídí), smyčka zpět nahoru — tudy skončí většina cyklů, proto levné.
- **Kolize -> vyhýbání** (jen teď): kandidátní ( $v, \omega$ ) nebo mezicíle, každý prorolluj, vyber bezkolizní s nejnižší cenou (DWA); když nic, brzdi.

Pointa: optimistická rychlá cesta, drahé vyhýbání jen když gate nahlásí zásah. Regulátor nech per-tick, rollout/predikci řeš vedle (samostatná třída s kopií modelu).

### 4 · Plánování cesty a vyhlazení

8-okolí gridu dává schodovité cesty (jen násobky  $45^\circ$ ). Neoptimalizuj schody, nejdřív se jich zbav a pak zaobluj:

- **Inflate gridu** (costmap) — cesta drží odstup a vznikne místo na zaoblení.
- **String-pulling** (line-of-sight) — greedy spojování vrcholů, kde je přímá spojka volná; LOS test = korydorový test (Collider) se šířkou robota. (Alternativa: any-angle search Theta\*/Lazy Theta\*.)
- **Zaoblení oblouky** (biarc) — vlož kružnicový oblouk tečný k oběma segmentům, poloměr omezený  $R_{\min} = \max(\text{Speed}, \max(\text{OrientationSpeed}))$  a dostupným odstupem; znovu ověř Coliderem.

Splajny (Bézier/B-spline) jsou hezké, ale nehlídají max křivost ani kolize -> víc práce. Oblouky s omezeným  $R_{\min}$  dají hladkost i soulad s dynamikou a padnou přímo do MotionArc + waypointů ( $x, y, \text{speed}$ ) s nenulovou průjezdovou rychlostí. Heavyweight varianta: elastic band / TEB.

### 5 · Costmap inflation

Inflate = pro každou volnou buňku spočítat vzdálenost k nejbližší překážce a zobrazit ji na cenu (robot se stane bodem = konfigurační prostor). Volba metody podle dat:

- **Řídký seznam kruhů** (tvůj Obstacle): vzdálenost je analytická ( $|\text{buňka} - \text{střed}| - \text{poloměr}$ ) -> orazítkuj kolem každé překážky předpočítané radiální jádro. Nejlevnější.
- **Hustý occupancy grid**: distanční transform — Felzenszwalb (exaktní,  $O(N)$ ), chamfer (Borgefors, aproximace), nebo multi-source vlnoplocha (ROS costmap\_2d).

Triky rychlosti: omez se na inflační poloměr; inkrementální „dirty“ update jen v okolí změn; předpočítaná cost LUT. Cena: lethal do robot\_radius, pak exponenciální doběh do  $R_{\inf}$ , dál 0.

### 6 · Reprezentace překážek

Bolesti pramení z toho, že jedna reprezentace dělá dvě práce: husté fúzované mapování volného prostoru vs. řídké sledování dynamických překážek. Rozděľ to:

- **Occupancy grid**: snadná fúze i clearing, ale binární + hrubý ztrácí přesnost v úzkých místech.
- **Seznam bodů**: přesný, ale musíš ručně řešit promazávání (raycast + index), fúzi (data association) a hlavně nemá volný prostor pro plánování koridoru.
- **Doporučené rozdělení**: lokální rolující ESDF/costmapa (fúze, clearing, koridory, clearance) pro plánování + řídké EKF-sledované kruhy (dynamika,  $k \cdot \sigma$ ) pro reaktivní Collider gate.

## 7 · ESDF — Euclidean Signed Distance Field

Pole, kde v každém bodě je uložena vzdálenost k nejbližšímu povrchu překážky.

- **Distance** — místo 0/1 ukládáš číslo = odstup; bohatší info.
- **Signed** —  $d > 0$  venku (volno),  $d < 0$  uvnitř překážky,  $d = 0$  na povrchu.
- **Euclidean** — pravá přímková vzdálenost (ne Manhattan/chamfer).

Ukládá se jako mřížka floatů, ale interpolací spojitě -> sub-buňková přesnost. Gradient  $\nabla d$  mívá od nejbližší překážky ( $|\nabla d| \approx 1$ , eikonál) -> využijí ho optimalizátory dráhy (CHOMP, TEB). Použití: clearance dotaz  $O(1)$ , náhrada inflace ( $\text{cost} = f(d)$ ), kolizní test dráhy, šířka koridoru.

## 8 · Budování ESDF

### 8.1 TSDF inkrementálně (fúze + clearing)

Z occupancy bys přišel o přesnost, proto stavíš z TSDF. Pro každý paprsek aktualizuješ voxely v pásu  $\pm r$  kolem zásahu ( $r$  = range povrchu,  $v\_range$  = range voxelu):

```
d_meas = r - v_range          # signed vzdálenost, ořež na [-tau, tau]
d = (w*d + w_meas*d_meas) / (w + w_meas)    # vážený průměr = fúze senzorů
w = min(w + w_meas, w_max)      # strop -> adaptace na dynamiku
```

- **d** — uložená (fúzovaná) signed vzdálenost; **w** — nasbíraná důvěra voxelu.
- **d\_meas** — co říká jedno měření ( $r - v\_range$ ); **w\_meas** — důvěra měření (konst.,  $1/z^2$ , úhel dopadu).
- **r** — truncation, pološířka pásu u povrchu (pár voxelů); **w\_max** — strop, aby mapa nezamrzla.
- Fúze = vážený průměr; clearing = voxely, kterými paprsek prošel před povrchem, jsou volné (space carving).

### 8.2 ESDF inkrementálně (raise / lower vlna)

TSDF dá přesné vzdálenosti jen v pásu  $\pm r$ ; ESDF je musí rozšířit do celého prostoru — čelní vlnou jen tam, kde se něco změnilo (Voxblox, FIESTA). Každý voxel drží dist a souřadnici nejbližšího povrchu (coord) — z ní se počítá pravý euklid.

```
for v in dirty:
    if v ve fixed pásu: esdf[v]={dist:tsdf[v], coord:v}; lower.push(v)    # seed z TSDF
    elif v se uvolnil:  raise.push(v)
```

RAISE: zneplatni voxely ukazující na zmizelou překážku, sousedy dej do lower

LOWER: pro souseda n: cand = || n - esdf[v].coord ||

```
if cand < esdf[n].dist: esdf[n]={dist:cand, coord:esdf[v].coord}; lower.push(n)
```

- **LOWER** — nová/bližší překážka: propaguje menší vzdálenosti ven.
- **RAISE** — clearing/vzdálení: invaliduje závislé voxely, pak je LOWER znovu naplní.
- Omez: maxDist (nešir dál než horizont), rolující okno kolem robota,  $w\_max$ . Pro UGV stačí 2D.

### 8.3 Felzenszwalbův separabilní EDT

Batch alternativa (přepočít lokálního okna každý cyklus). Kvadrát euklidovské vzdálenosti se separuje na osy, takže 2D = dva 1D průchody: nejdřív po sloupcích, pak po řádcích s výsledkem prvního průchodu jako počáteční cenou.

1D transform  $D(p) = \min_q [ (p-q)^2 + f(q) ]$  je **spodní obálka parabol** (jedna na semeno  $q$ , vrchol  $(q, f(q))$ ). Všechny mají stejné zakřivení -> protnou se právě jednou -> obálka se spočítá jedním průchodem v  $O(n)$ :

# FÁZE A: postav obálku ( $v[]$  = vrcholy,  $z[]$  = hranice)

for  $q = 1..n-1$ :

```
s = prousecik(q, v[k])          # s = ((f(q)+q^2)-(f(v[k])+v[k]^2)) / (2q-2v[k])
while s <= z[k]: k--; s = prousecik(q, v[k])    # vyhoď překrytou parabolu
k++; v[k]=q; z[k]=s; z[k+1]=+inf
```

# FÁZE B: vyhodnot' pro každé  $p$

$j = 0$

for  $p = 0..n-1$ :

```
while z[j+1] < p: j++
D(p) = (p - v[j])^2 + f(v[j])
```

- Každá parabola se jednou přidá a nejvýš jednou vyřadí ->  $O(n)$  na řádek/sloupec, celkem  $O(N)$ , exaktní euklid.

- Sub-voxel: seeduj  $f(q)=\delta^2$  z TSDF (interpolovaná nula). Signed: dvakrát (venek + doplněk) nebo znaménko z TSDF.
- Rozšíření do 3D: přidej průchod po ose z.

## 9 · Otevřené body a další kroky

- Zrychlení v predikci: dělení fáze brzdění na K oblouků (dnes jeden oblouk na fázi).
- Rozšíření regulátoru na víc waypointů + rollout jako samostatná predikční třída.
- Vyříznutí safe corridoru z ESDF (IRIS / safe flight corridor) pro hladký průjezd úzkým místem.
- Geo adaptér WGS84 <-> lokální metry pro napojení modulu Colider na OSM stranu.

---

Shrnutí návrhové diskuse — technický dokument. Bez implementačních závazků nad rámec modulu Colider, který je hotový a otestovaný.